CPR E 3810 Toolflow Manual

October 2025

1 Overview

The enclosed toolflow implements a testing and synthesis workflow using Modelsim (Questasim) and Quartus Prime. Both the testing and synthesis portions are necessary to ensure proper completion of all phases of the CPR E 3810 Course project - a riscv processor.

This tool is divided into 2 tools: 'test' and 'synth'. The test tool optionally compiles your processor, simulates 1 or more assembly programs on RARS, initializes a Modelsim simulation with your processor, loading in a test program, and then compares the simulation results of your processor with RARS.

The synth tool synthesizes your processor for an Altera FPGA, and then outputs a timing summary that should indicate your critical path.

2 Usage

3810_tf.sh: This runner provides some environment setup and actually calls the test scripts. It may be run like

```
./3810_tf.sh <new|test|synth|submit> [options]
```

2.1 Running 3810_tf.sh new

The first mode for running the tool flow, new, initializes a new project in the directory "proj". This directory contains several sub-directories.

- riscv
  - This directory will contain all of your requires riscv source code for the project. Several sample programs are provided.
  - Add additional riscv test files here as needed.
- src
  - This directory contains your processor source code. You may structure the directory however you wish, however, the base structure we provide is as follows.
    - riscv_types.vhd
      - This file is guaranteed to be compiled first by the toolflow
      - Place all type declarations and global constants in here
      - You may include this file in any other source
    - TopLevel/riscv_Processor.vhd

        - This file contains the skeleton processor. Be mindful of all of the required signal names.
    * TopLevel/mem.vhd
        - This contains a complete memory entity. This is the same module tested in Lab 2

- test
  - Place your VHDL test benches in here.
  - Required for submission

It is recommended that your team use some sort of version control. If you elect to use git, initialize a git repository inside of the proj directory.

```
cd proj
git init

cat << EOF | tee README
CPR E 3810 Project 1
Teamate 1 Name <email@iastate.edu>
Teamate 2 Name <email@iastate.edu>
Teamate 3 Name <email@iastate.edu>
EOF
```

2.2 Running 3810_tf.sh test [options] file...

This command is what you will use the majority of the time to test your processor's integration functionality. The command will compile your code, assemble one or more riscv programs, simulate the riscv programs on both RARS and your processor, and then compare the output. This framework is very similar to the one that will be used to grade the correctness of your processor.

- `--help` Prints a help message displaying these options
- `--summary #` Print a tabular output
- `--max-mismatches #` Allow # mismatches (default 30)
- `--nocompile` Don't recompile VHDL (default compiles)
- `--sim-timeout #` Number of seconds to simulate for (Default 30)
- `--config CONFIG` Selects an alternate system configuration

```
./3810_tf.sh test proj/riscv/addiseq.s
```

Example 1: Running 1 file

```
./3810_tf.sh test  --nocompile proj/riscv/addiseq.s
```

Example 2: Running 1 file, no compile

Examples 1 and 2 run a single file, proj/riscv/addiseq.s. The results of this run will be stored in the output directory.

In general, the output directory contains all of the output, intermediate files, and logs of a single run. The toolflow will create a new directory for each test. However, after re-running a test, the old output will be thrown away.

```
./3810_tf.sh test proj/riscv/*
```

Example 3: Running a directory

The test_failures.txt file generated for batch runs may also be fed back into the toolflow to rerun failures.

```
./3810_tf.sh test @output/test_failures.txt
```

Example 5: Running failures via a file

The above command can be run again and again, with each run only running the failures, then overwriting the test_failures.txt file with new failures.

2.3 Running 3810_tf.sh synth

Synthesis can be run via -

```
./3810_tf.sh synth
```

Example 6: Running Synthesis

Synthesis output, such as timings files, can be found in the temp directory. Be aware – synthesis may take several hours, particularly for Project 1.

2.4 Running 3810_tf.sh submit

Lastly, configure your proj/ file structure to match the submission guidelines. This should look like:

```
proj
|---riscv
|      |---file.s
|      \---file2.s
|---src
|      |---proc...
|---test
|      |---tb_1.vhd
|      \---tb_2.vhd
\---report.pdf
```

```
./3810_tf.sh submit
```

Example 7 : Generating Project 1 submission

Your submission will be sanity checked, and a final zip and report pdf will be placed in the submissions directory. Please make sure to upload both the zip and the report pdf to Canvas.

For project 2, add the option 'hw' or 'sw' to create a submission for either the hardware scheduled or software scheduled pipeline. Ultimately, you should create 2 zip folders, and submit both.

```
./3810_tf.sh submit sw
```

Example 8 : Generating Project 2 Software Pipeline submission

```
./3810_tf.sh submit hw
```

Example 9 : Generating Project 2 Hardware Pipeline submission

A Git

A.1 Motivation

While students are free to share code amongst their team however they please, one common solution used in many development projects is Git. If you are not aware of the fundamentals of Git, there are plenty of online resources that teach the fundamentals.

A.2 Initializing your repository

First, navigate to git.ece.iastate.edu. Click "New Project", followed by "Create Blank Project". Name your project, set visibility to "private", and **Deselect "Initialize with Readme"**. After your project has been created, record the repository URL by clicking "clone" and copying the HTTPS URL.

Next, on the lab machine or VDI, open terminal and first configure Git locally if you have yet to do so using –

```
git config --global user.name "Your Name"
git config --global user.email "netID@iastate.edu"
```

Next, use the "CD" command to change directories into the the cpre3810-toolflow directory. Follow the steps in Section **??** to create your project directory. Next, add your repository to Git by initializing a new repo in the proj directory. For example:

```
./3810_tf.sh new
cd proj
git init
git add .
git commit -m "Initialize repo"
git remote add origin <Repository URL>
git push -u origin master
```

Lastly, your teammates may clone the repo into the toolflow directory as follows. Make sure to first navigate inside of the cpre3810-toolflow directory using cd.

```
git clone <Repository URL> proj
```

The newly cloned processor can then be run like normal with the toolflow.

B Config Files

B.1 Motivation

To run the toolflow on a non-lab machine, you may need to use an alternate configuration. Configuration is handled via a file config.ini, placed in the root of the toolflow directory. A sample of this file may be found in the internal directory.

B.2 Format

The file consists of one or more configurations organized into sections. Sections are denoted using brackets, "[MySectionTitle]". Key value pairs are then inserted underneath the section block using the form "key = value". There are 3 user configurable parameters - "modelsim_paths", "quartus_paths", and

"needs_license". In general, "needs_license" should always be set to false unless you are using a univerisity licensed version of quartus. The two path parameters each accept a list of quoted *nix style paths, separated by commas and enclosed in brackets. If multiple paths are provided, then each path will be checked in order until a valid option is found.

```
[Config]
modelsim_paths=["/path/to/modelsim/bin", "/path2/to/
    modelsim/bin"]
quartus_paths=["/path/to/quartus/bin"]
needs_license = false

[AnotherConfig]
modelsim_paths=["/anoother/path/to/modelsim/bin"]
quartus_paths=["/another/path/to/quartus/bin"]
needs_license = false
```

Example 10 : Example Configuration

B.3 Using the configuration

To use a non-default configuration, you must specify to both the test and synthesis frameworks which configuration you would like to use. The name of the configuration that is passed to the toolflow must match a section in the config.ini file.

```
./3810_tf test --asm-file file.s --config Config
./3810_tf test --asm-file file.s --config AnotherConfig

./3810_tf synth --config Config
./3810_tf synth --config AnotherConfig
```

Example 11 : Running with New Configurations